

Try Claude Code

Engineering

How the Claude Code team does code review

How do you review code at the volume agents produce it? On the Claude Code team, once we had high coverage and deep reviews, we started optimizing for the minimum amount of human review time that doesn't raise reverts or incidents.

AUTHOR
└ Addy Osmani, Morgan Westlee Lunt, and Qing Wang

PUBLISHED
└ Oct 15, 2026

READING TIME
└ 14 min

We've said before that Claude writes about 90 percent of Claude Code's own code. So how do we review code at the volume we produce it? Does anyone read code anymore? And if something breaks, who catches it?

This post covers the code review flow on the Claude Code team and how it has evolved. From June to September, merged pull requests to Claude Code's codebase more than doubled while the share that had to be reverted fell by more than 40 percent.

We didn't remove review. But we did move most of the reading to machines, optimized for human time and attention, and invested heavily in making it fast and easy to undo mistakes.

RETHINKING THE PURPOSE OF CODE REVIEW

When agents write code, the number of changes goes up, fast. In Anthropic's monorepo, changes landing per day went from about 2,500 in May to more

than 30,000 in September this year.

Claude Code lives in its own repository, separate from the monorepo, but it's seeing a lot of pressure too. When you are shipping this fast, manual, line-by-line code review is incredibly difficult if not impossible.

Code review does two jobs:

1. Find and fix issues before they ship
2. Build shared context and codebase understanding across the team

This article mainly focuses on finding and fixing issues, but we will come back to the second job near the end.

We think "did a human read the code?" is an imprecise proxy for "what could go wrong with this change, and which check—automated or human—is the right one to catch it?"

Asking that question also changed *when* review happens. Before, review was a gate at the end of the road: write the code, open a PR, wait for your colleague.

Now, review runs continuously, more like spellcheck while you write. Most changes have been reviewed and fixed by Claude at least once before a human is asked to look. This catches most routine mistakes, and saves team attention for higher-level questions and scrutiny.

THE SCARCE RESOURCE: HUMAN TIME AND ATTENTION

We want to preserve human time and attention for issues only humans can solve and make sure automated checks catch the rest. At our current rate of change, there is still an incredible amount of work for humans to do, and our team encounters new and novel challenges almost daily.

Human time covers both sides of review's cost: the time people spend reviewing, and the time they spend fixing what bad code breaks. Cutting the first doesn't help if the time just shows up in the second.

So the goal is that most pull requests in the Claude Code repos (the CLI, the desktop and web apps, and the mobile apps) merge with at most one human in the loop, who can be the author, with no increase in reverts or incidents. To

measure human time, we track human approvals per PR (median and 95th percentile), human comments, and how long a PR waits on a person. Revert and incident rates are the guardrail: if they rise, we haven't saved human time, we've only moved it.

THE LIFE OF A CHANGE

A typical change, from an engineer's laptop (or a Claude session in Slack), goes through multiple layers of code review.

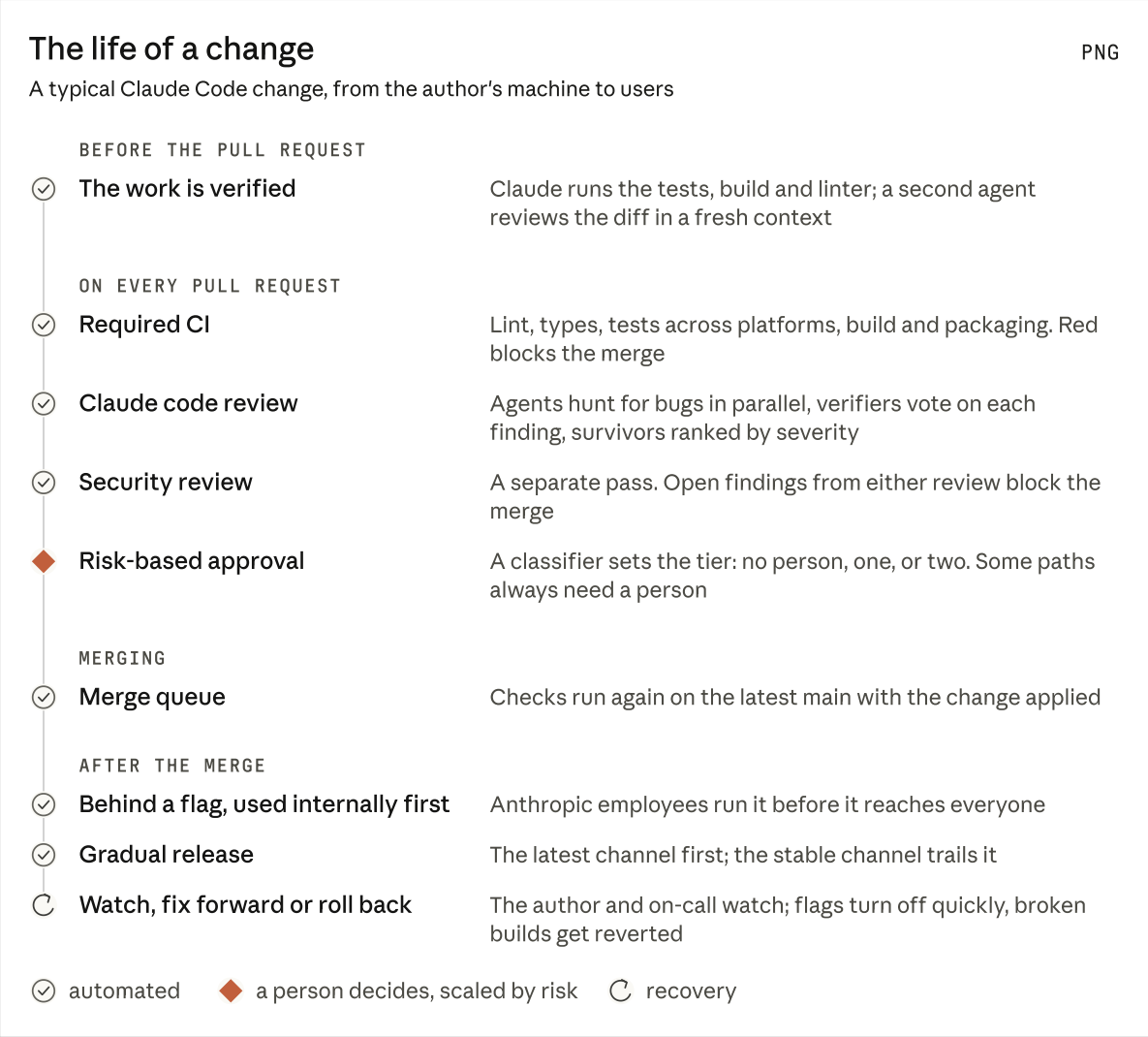


FIG A Most of the path is automated; a person's attention goes where the risk is

LAYER BY LAYER

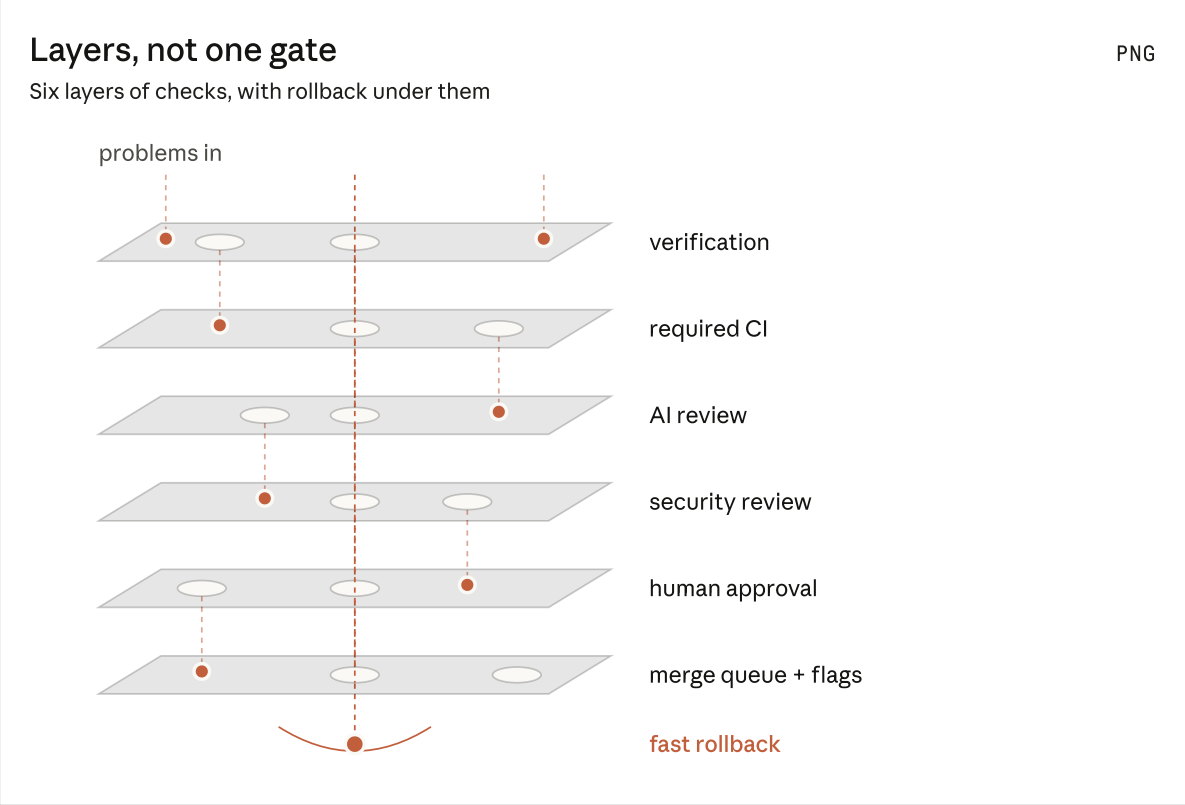


FIG B Each layer catches some problems and misses others. Few get through all of them, and rollback handles the rest.

1. The work is verified before anyone else looks

Most review now happens before a PR exists. We give Claude something it can run—like tests, a build or a screenshot to compare against the design—and ask it to show the output instead of saying it's done. Then a subagent reviews the diff with fresh context. Both habits come from our best-practices guide.

Before opening a PR, many of us also run a few internal skills: one hunts for bugs, one simplifies the diff, and one checks the behavior end to end. UI changes also get checked against our design guidelines.

Boris Cherny, who created Claude Code, has called giving Claude a way to verify its work probably the most important thing for getting good results. He estimates it makes the final result two to three times better, and writes, "Verification looks different for each domain. It might be as simple as running a bash command, or running a test suite, or testing the app in a browser or phone simulator."

When the automated reviewer keeps flagging the same thing, we fix it at the source: in the skill, with a lint rule, or with updates to instructional files such as:

- `CLAUDE.md` tells Claude what the project is: the stack, the conventions, the goals and the non-goals. Every review loads it, so in a large codebase we keep it short and point to other docs from there. The reviewer also flags code that breaks a convention in it, and changes that make a line in it out of date.
- `REVIEW.md`, written by code owners, tells the reviewer what matters in their part of the code. We check separately whether the reviewer actually follows it. The good ones are short. They say what failure looks like in that repo (for docs it's "it lies," for config it's "it misroutes"), cap the nits, list paths to skip, and ask for a file and line as evidence instead of a guess from a name. They also ask for a summary that starts with a count and says plainly when there's nothing to fix.

2. AI review on every pull request

Every pull request gets a Claude review before a person looks. It reads the diff against the whole repo and the PR's stated intent, and asks: does the change solve its problem, fit the system, or break something else?

On a bug fix, it looks for the root cause. The public version, Code Review for Claude Code, is now generally available on Team and Enterprise plans, and `/code-review` runs a lighter version locally.

When we launched the research preview, the share of internal PRs getting substantive review comments went from 16% to 54%. Most PRs over 1,000 lines get findings, and most small ones come back clean. Admins set each repo to Quick, Standard, or Deep. Deep is worth it where one missed bug costs more than the extra review time.

Findings get verified by other agents before humans review them. Several agents look for bugs at the same time. Then a separate group votes on each finding, and it only gets posted if enough of them agree it's real. Findings that are only about style, diagnostics, or cleanup get dropped. We A/B tested those filters on our own PRs before keeping them. What's left shows up as inline

comments sorted by severity. Each is tagged Important, Nit, or Pre-existing, so a bug the PR didn't introduce doesn't look like the author's fault.

Every finding has to be answered. You either fix it or resolve the thread with a reason. Often an agent picks up the fix overnight, so the author comes back to a PR that's already been through another round.

3. Security review that can block

Security gets its own automated review too. A PR can't merge while any finding from the bug review or the security review is still open. You either fix it or close the thread with a short reason, which is how you clear a false positive.

If a later commit fixes the issue, the finding clears on its own. You can set up the same thing with the open-source security review GitHub Action or the `/security-review` command. Setup is covered in automated security reviews in Claude Code.

4. Automated checks that block

The usual: lint, type checks, tests on every platform, and a build. (We wrote about scaling CI for this volume here.) A typical CLI pull request may run dozens of checks before anyone looks at it. The PR template asks how the change was tested and how to undo it. Claude can fill that in, but the person who asked for the change signs off on it.

5. Human approval, scaled by risk

A docs fix and an auto-updater change carry different risk, so they get different review.

RISK	EXAMPLES	HUMAN APPROVALS
Low	Docs, copy, internal-only features, additive changes behind a flag	None required once automated review and security checks are green

RISK	EXAMPLES	HUMAN APPROVALS
Medium	Ordinary product changes, and changes to sensitive areas such as caching, the auto-updater, or system prompts	One person
High	The highest-risk code, plus a small set of paths that always need two people	Two people

The change sets the tier, not the author. A classifier reads the change and picks its risk tier. A second classifier asks whether this particular change needs another person even when its tier says it doesn't. Some paths always need a person, anything owners have listed as sensitive and any CI workflow file.

Only trusted authors get the lighter paths. Only authors on the repo's trusted list get the lighter paths, mostly the team that owns it. Everyone else needs a second person to approve. An agent's approval never counts as a person's, either. Automated review can clear a low-risk change, but it can't fill a slot that calls for a human.

Whoever prompted the change is on the hook for it. On an agent-written PR, the person directing the agent counts as one of the required reviewers, the same as if they'd typed the code themselves.

An approval covers one specific commit. If new commits land that the system can't vouch for, like an agent's push or a merge someone resolved by hand, the approval has to be given again.

6. Merge queue, flags and a gradual release

The merge queue re-runs every check against main with the change applied, so two PRs that pass separately can't combine into a broken main. New behavior usually ships behind a flag to Anthropic employees first, and our heavy use often surfaces regressions before users do. Public releases go to the `latest` channel first. The `stable` channel trails it.

7. Fast detection, fast rollback

Some mistakes will still get through, so the last layer is about catching them fast. Flags turn off without a deploy. Claude Code has a weekly on-call rotation, and incident reviews ask which layer should have caught the issue.

Noticing a bad merge still depends on the author and on-call as well as the revert and incident dashboards. We don't yet have circuit breakers that tighten approval when those numbers move; engineers here want them before more code moves to zero-human merges.

One colleague said slowing down does lower your error rate, but mostly because you're doing less of everything. We'd rather get faster at noticing and fixing problems.

HOW WE MAKE THE REVIEWER BETTER

Our code review process is constantly improving in a loop.

Any incidents are now an eval. The team collected past PRs that caused incidents, with their fix-forwards and reverts, and had engineers who know that code confirm each bug and where it lived. The reviewer runs against that set daily.

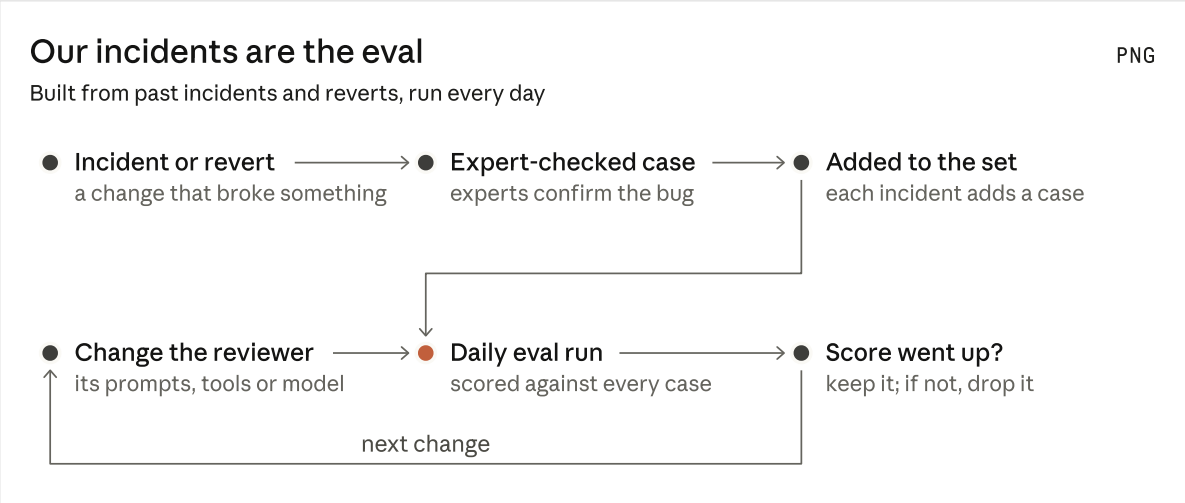


FIG C The eval grows with every incident, and every change to the reviewer is scored against all of it.

We measure noise by watching what people do with the comments. It's easy to catch more bugs if you don't mind flagging a lot of non-bugs, so we need a

check on that. We sort every comment the reviewer leaves by how the person on the PR responded: they fixed it, said it was wrong, said the behavior was intentional, or decided it didn't matter. We also run the reviewer on PRs we know are clean and count how often it reports something that would have led someone to make a harmful change. Our priority, written down, is catching real bugs first and cutting review rounds second.

Noise costs more when the author is an agent. A person can wave off a nit in seconds. An agent fixes it, which means another commit and another round of review.

Review has to converge. An agent author and an agent reviewer can keep each other busy indefinitely. One CLI PR went through more than fifty rounds of review and fixes overnight before its author stopped it.

When we ran git blame on what the reviewer kept flagging, about 59% of it was code that had been there since the reviewer's first pass. The reviewer was rarely finding new bugs, it was finding a few more old ones each round.

We made it worse ourselves. We'd gotten used to running several rounds by default, since each pass catches something new. This can be worthwhile on a risky change, but not on every PR. The team now tracks rounds-to-pass alongside recall, and changed re-review:

- **A round cap.** After three rounds, the reviewer asks the author to step back and reconsider the approach.
- **Diff-only re-review** after the first few passes.
- **Flag the class of bug,** so the author fixes every instance, not one line.
- **No repeats** of findings already posted.

Review agents also run nightly over existing code, helping us catch old bugs before they show up somewhere else.

Review that converges PNG

Internal pull requests, before and after the convergence changes

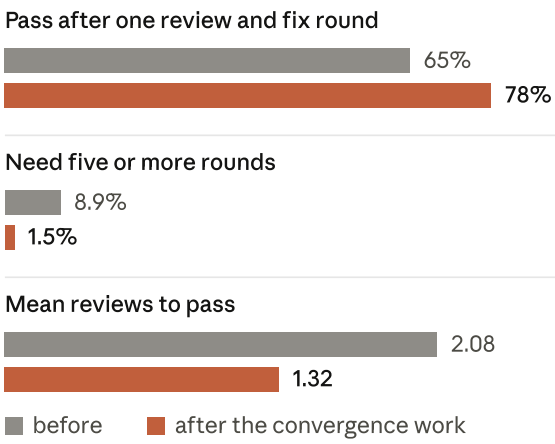


FIG D Fewer review rounds per internal pull request after the convergence changes

WHERE HUMANS STAY IN THE LOOP

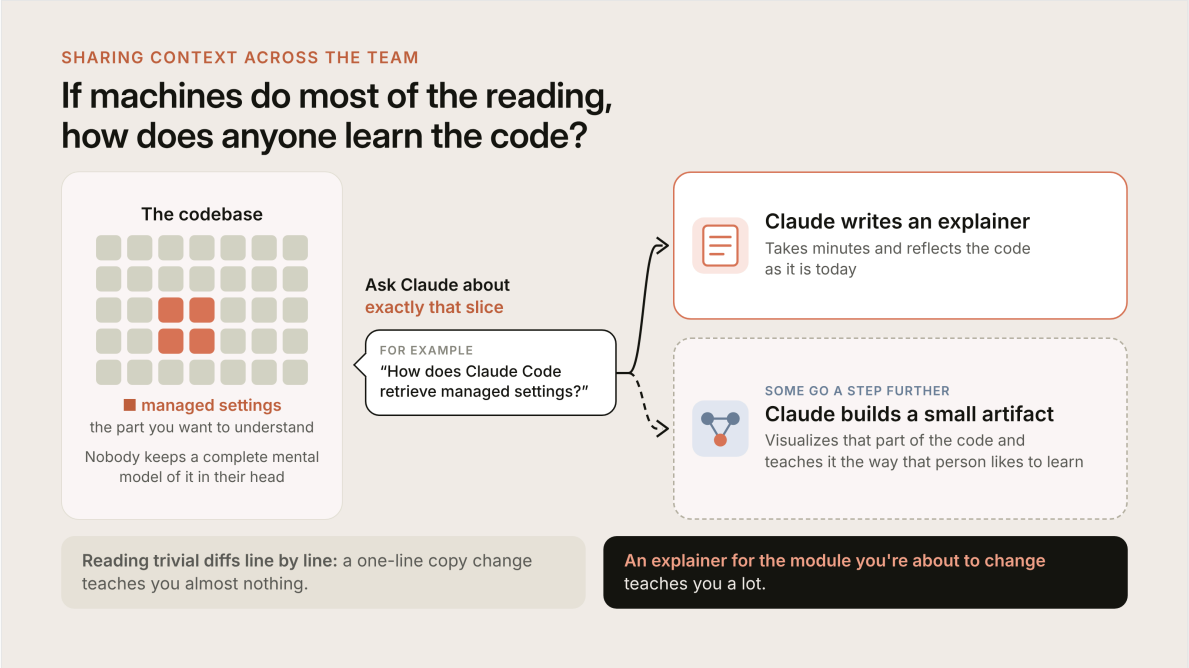
Pulling a senior engineer out of deep work to catch a routine mistake costs far more than an automated review. People on the team are still responsible for:

- **Deciding whether to build it.** Direction, taste and scope. One team that declared its review bottleneck solved still kept human review for changes from outside the team, so someone asks "should we even do this?"
- **Reviewing risky code** such as billing, the auto-updater and CI workflows.
- **Reviewing architecture** that sets a pattern others will copy.
- **Reading the evidence.** Test output, the command that ran and what it returned, a screenshot, the reviewer's findings. If you can't say what the checks would miss, ask for a real review rather than a quick approval.
- **Owning the outcome.** A person is accountable for every change that ships, whoever typed it.

SHARING CONTEXT ACROSS THE TEAM

Code review's second job has always been to spread codebase understanding. Reading a teammate's diff is how you learn what changed and how the pieces fit. If machines do most of the reading, how does anyone learn the code?

We don't have a formal process for this. Nobody on the team keeps a complete mental model of Claude Code in their head, and we don't think anyone needs to. When someone wants to understand a part of the codebase, they ask Claude to write an explainer for exactly that slice. It takes minutes and reflects the code as it is today.



Some of us go a step further and have Claude build a small artifact that visualizes that part of the code and teaches it the way that person likes to learn.

If the goal is to spend human attention well, this beats reading trivial diffs line by line. A one-line copy change teaches you almost nothing. An explainer for the module you're about to change teaches you a lot. Our culture of defaulting to working on open Slack channels helps too, since you see what others are building while they build it.

"YOU LAND IT, YOU OWN IT"

If you land a small or medium change and something still goes wrong, it's a team norm that you usually fix it. A few things keep this from going badly:

- **Changes are small,** so they're easy to review, debug and revert.

- **Everything is reversible.** Flags, reverts and staged releases undo most mistakes in minutes.
- **Blast radius sets the bar.** Expensive code gets more people and more checks.
- **Regressions surface fast** through telemetry and internal use.
- **It isn't blame.** Owning a fix means you have the most context; incident reviews look at the system.

Ownership is also shared. Much of the team works in open channels where anyone can see what others are building, pick up a thread, or point a Claude at a gap, and feedback comes from people who don't own the code too. That openness is a big part of how quality stays high without a gate on every line.

WHEN WE CHANGE DIRECTION

Agents make it cheap to start things, so some work shouldn't land. Usually someone in the channel asks why, points at an earlier decision, or suggests a simpler route, and the author closes or reshapes the PR.

Since the code was fast to write, closing the PR doesn't feel like a loss. The trick is to raise a different direction in the thread early, not as a blocking comment on a finished PR.

HOW YOU CAN ADOPT THIS PROCESS

Most of the pieces to do this with your team exist today.

Before the pull request

- Give your agent checks it can run, listed in a short `CLAUDE.md`, plus hooks that run formatters and linters.
- Run `/code-review` and `/security-review` locally, or a fresh-context subagent review.
- Add a `REVIEW.md`, and move repeated review comments into it.

On every pull request

- Require CI, and add "Testing", "Risk" and "How to revert" to your PR template.
- Add an AI reviewer (Code Review for Claude Code, or the Claude Code GitHub Action) on one repo, tune `REVIEW.md` for a few weeks, then widen.
- Add automated security review, and require every review thread resolved before merge.
- Match review effort to risk: a quick pass on routine changes, a deep one where a missed bug is expensive (Quick, Standard or Deep in Code Review).
- Cap review rounds; after the first few, re-review only the new diff.

Approval

- Write down risk tiers, set from the change rather than the author; start strict and loosen with evidence.
- Keep the lighter approval paths for authors you trust.
- Use CODEOWNERS and branch protection so auth, billing, CI and release workflows, and installers always need a person.
- Never let a bot's approval replace a required human's; dismiss approvals on new commits.

Merge and after

- Use a merge queue, feature flags, internal use first, staged releases and one-step rollback.
- Turn incidents and reverts into an eval set, and rescore the reviewer whenever its prompts, tools or model change.
- Measure human time, with revert and incident rates as the guardrail.

WHERE THIS IS GOING

Code Review's announcement said approving a pull request is still a human call; the product posts findings, not approvals. Our own repos go further,

letting owners decide which low-risk changes merge on automated checks alone, and that line keeps moving.

When a review misses something, we ask what the model didn't have in front of it and put the fix into a `CLAUDE.md`, a skill or a new check. The goal hasn't changed: less human time per change, and no more reverts than before.

If your team lets agents open pull requests, we'd like to hear which checks you've automated, which still need a person, and what you think we have wrong.

With thanks to Emma Webb, Michael Segner and Amie Rotherham for their reviews.

SHARE

└ X.com
└ LinkedIn
└ Email
└ Copy URL
└ Copy markdown

Related posts

ALL AGENTS ENGINEERING PLAYBOOKS SKILLS TUTORIALS

Oct 08, 2026	Building effective agent automations	9 min
Oct 06, 2026	Claude Code in the cloud: a field guide to clou…	18 min
Oct 01, 2026	Getting started with Claude Code mods	11 min
Sep 28, 2026	Automating eval design and hillclimbing with C…	12 min
Sep 28, 2026	Building with Claude Sonnet 5.5	9 min

LOAD MORE

claude.dev

RESOURCES

Claude Academy

Claude Code

Community

Console

Developer docs

CONNECT

Discord

Meetups

Reddit

X.com

GET THE NEWSLETTER

Product updates, how-tos, community spotlights, and more. Delivered monthly to your inbox.

Enter your email

Please provide your email address if you'd like to receive our monthly developer newsletter. You can unsubscribe at any time. Read Anthropic's Privacy Policy.

ANTHROP\C

© 2026 Anthropic PBC

Privacy Policy Terms of Use Privacy choices